

第10週: モダン開発へ移行しよう (Next.js)

～ Vanilla JS からプロの道具へ乗り換え！ ～

本日のメニュー

1. レゴブロックの考え方: なぜNext.jsなの？
2. フォルダの冒険: プロジェクトの中身を知ろう
3. JSXのルール: HTMLとJavaScriptの合体
4. 爆速デザイン: Tailwind CSS 入門
5. 共通の枠組み: レイアウトを作ろう
6. 最初のページ: `page.tsx` を書き換える
7. 実践タイム: ダッシュボードをNext.jsに引っ越そう！

1-1. Next.js：最強の「部品組み立て」ツール

これまでの開発と、これからの開発は何が違うのでしょうか？

- **これまで (Vanilla):**

- 1枚の長い長いHTMLを頑張って書く。
- JSで「h1を探して色を変えて…」と細かく指示を出す。

- **これから (Next.js):**

- ボタン、カード、メニューなどを「部品（コンポーネント）」として作る。
- 部品をレゴブロックのように組み合わせて、ページを作る！

2-1. プロジェクトの中身を覗いてみよう

`create-next-app` というコマンドを打つと、たくさんのファイルが現れます。

- `app/`: ここが皆さんの「メイン作業場」です。
- `app/page.tsx`: ブラウザに表示される「中身」そのもの。
- `app/layout.tsx`: ページ全体の「共通の枠組み（ヘッダーとか）」。
- `public/`: 画像やアイコンを入れる場所。

3-1. JSX：HTMLが進化した姿

Next.js（React）では、JavaScriptの中に直接HTMLのようなものを書きます。これを **JSX** と呼びます。

- 微妙に違うルールに注意！

- `class` → `className` と書く。
- 閉じタグは絶対に必要！（例: `<img />`, `<br />`）
- `{ }` を使えば、HTMLの中でJSの変数が見える。

4-1. Tailwind CSS : クラス名だけでデザイン

CSSファイルを作らず、HTMLタグに直接「意味のある名前」を書いてデザインします。

```
// 青い背景、白い文字、角を丸く、余白を4
<button className="bg-blue-500 text-white rounded-lg p-4">
  ボタン
</button>
```

- メリット:
 - CSSファイルを往復しなくていい。
 - クラス名に悩まなくていい。

5-1. レイアウト：みんなの「外枠」

すべてのページで共通のパーツ（ヘッダーやサイドバー）は、`layout.tsx` に書きます。

- 役割:
 - ページを移動しても、サイドバーが消えないようにする。
 - 全体の背景色などを一括で決める。

```
export default function RootLayout({ children }) {  
  return (  
    <html>  
      <body>  
        <nav>サイドバー</nav>  
        {children} { /* ここに各ページの中身が入る！ */ }  
      </body>  
    </html>  
  );  
}
```

6-1. 【やってみよう 1】最初のページを書き換えよう！

1. `app/page.tsx` の中身を全部消す。
2. 自分の名前を表示するだけのシンプルなコードに書き換える。

```
export default function Home() {  
  return (  
    <main className="p-10">  
      <h1 className="text-4xl font-bold text-orange-500">  
        Next.js の世界へようこそ！  
      </h1>  
      <p>これは私の最初のモダンWebページです。</p>  
    </main>  
  );  
}
```

✅ **成功イメージ:** ブラウザが自動で更新されて、新しい文字が出たら成功！

7. まとめ：ダッシュボードをお引越し！

これまで作ってきたHTMLの中身を、`app/page.tsx` にコピーして、モダンな書き方に直しましょう。

- 目標:

- HTMLタグをJSX (`className` など) に書き換える。
- Tailwind CSS を使って、これまでのデザインを再現する。
- 画像は `public/` フォルダに入れて読み込む。
- コンパイルエラー（赤い画面）が出ないように閉じタグをチェック！

8. 迷った時のチェックリスト

1. `className`: `class` と書いていませんか？
2. 閉じタグ: `<img>` や `<br>` は `<img />` や `<br />` になっていますか？
3. リターン: `return ( ... );` のカッコの中にHTMLが入っていますか？
4. 小文字: HTMLタグ (`div`, `p` など) は小文字で書き始めていますか？

9. 今週の宿題 (Homework)

1. ダッシュボードの「見た目」をNext.jsで完璧に再現する。
2. **チャレンジ:** Tailwind CSS の公式ドキュメントを見て、一つ新しいデザイン設定を試してみる。
3. 来週のために、「どこを部品（ボタンとかカード）として独立させようかな？」と目星をつけておく。

✓ 最後に確認:

- ☐ 画面がモダンなツールで表示できたかな？
- ☐ Tailwind CSS の便利さを少し感じられたかな？
- ☐ JSX の書き方に慣れてきたかな？